

Assignment 10

This is the second exercise sheet dedicated to quadrature. The first two exercises focus on Gaussian quadrature rules and related ones, while the third exercise is about Simpson's quadrature rule.

Graded exercise

Exercise 34 (Computing nodes and weights for Gauss quadrature rules)

We have seen in class that the nodes of a Gaussian quadrature rule coincide with the zeroes of orthogonal polynomials. Usually these are computed numerically (and then possibly tabulated for later use). In this exercise, we see how to perform this task.

We first need an auxiliary result, corresponding to Theorems 3.6.20 and 3.6.21 in the book (J. Stoer and R. Bulirsch, *Introduction to Numerical Analysis*, 3rd edition) - where you can also find the proofs, in case you are interested.

Theorem. Let $\{p_n\}_{n \geq 0}$ be a family of orthogonal polynomials satisfying $p_0(t) \equiv 1$, $p_1(t) = (t - \delta_1) - \gamma_1^2$ and the recursion

$$p_n(t) = (t - \delta_n)p_{n-1}(t) - \gamma_n^2 p_{n-2}(t), \quad n \geq 2, \quad (1)$$

for some real sequences $\{\delta_n\}_{n \geq 1}$, $\{\gamma_n\}_{n \geq 1}$. Then the roots $c_1, \dots, c_n$ of the n -th orthogonal polynomial p_n , $n \geq 2$, are the eigenvalues of the tridiagonal matrix

$$J_n = \begin{pmatrix} \delta_1 & \gamma_2 & & & \\ \gamma_2 & \delta_2 & \cdot & & \\ & \cdot & \cdot & \cdot & \\ & & \cdot & \cdot & \gamma_n \\ & & & \gamma_n & \delta_n \end{pmatrix}. \quad (2)$$

When using these roots as nodes of a quadrature formula, the associated weights $b_1, \dots, b_n$ are given by

$$b_i = \frac{2\tilde{b}_i}{\sum_{i=1}^n \tilde{b}_i}, \quad \text{where } \tilde{b}_i = (v_1^{(i)})^2, \quad i = 1, \dots, n,$$

and $v_1^{(i)}$ denotes the first component of the i -th eigenvector of the matrix J_n .

This result tells us that we can compute the nodes and weights of a Gaussian quadrature rule by solving an eigenvalue problem. Since the weights have been self-normalized, it does not matter which normalization is used to define the eigenvectors. From now on, we focus on Gauss-Legendre quadrature rules.

The Legendre polynomials follow the recursion

$$\mathcal{L}_n(t) = \frac{2n-1}{n} t \mathcal{L}_{n-1}(t) - \frac{n-1}{n} \mathcal{L}_{n-2}(t) \quad n \geq 2, \quad (3)$$

with $\mathcal{L}_0(t) \equiv 1$, $\mathcal{L}_1(t) = t$ (see lecture notes).

- (a) Show that the sequences $\{\delta_n\}_{n \geq 1}$, $\{\gamma_n\}_{n \geq 1}$ in (1) for the Legendre polynomials are given by $\delta_n = 0$ for all $n \geq 1$ and $\gamma_n = \frac{n-1}{\sqrt{4(n-1)^2-1}}$, $n \geq 1$.

Hint: The difference between (3) and (1) is due to a different normalization used for the polynomials. This means that $p_n(t) = \alpha_n \mathcal{L}_n(t)$ for some numbers α_n , $n \geq 1$. Use this in (3) to obtain a recursion for $\{p_n\}_{n \geq 2}$. You will obtain a relationship between α_n and α_{n-1} for all $n \geq 1$ by imposing the coefficient of p_{n-1} to be equal to 1, as required from (1). Using also this in the recursion you obtained for $\{p_n\}_{n \geq 2}$ will lead you to the expression for δ_n and γ_n , $n \geq 2$ (the case $n = 1$ can be obtained by looking at $\mathcal{L}_1$ directly).

- (b) Write a PYTHON routine that takes in input the number n of nodes that we want for the Gauss-Legendre quadrature rule and returns the nodes and the weights. To compute these, use the auxiliary result given at the beginning of the exercise. Check your routine by calling it on a script for getting $n = 4$ nodes and weights, and compare the results with one of the built-in functions, either `numpy.polynomial.legendre.leggauss` or `scipy.special.roots_legendre`. For performing the check, the script should print on screen both the nodes and weights that you obtain with your routine and those from one of the two built-in PYTHON functions. It does not matter if the order of your nodes and those from PYTHON do not coincide, as long as the ordering of the associated weights is consistent.

Hint: Use the information from task (a) to allocate the matrix J_n as from (2). In your routine, to obtain the eigenvalues and eigenvectors of J_n , you can use the built-in function `numpy.linalg.eig`.

- (c) Write a script that computes numerically the integral $\int_{\frac{\pi}{2}}^{\pi} \sin(x) dx$ using your routine from (a) with $n = 4$ and prints the result on screen. The exact value of the integral is 1.

Other exercises

Exercise 35 (Gauss-Radau and Gauss-Lobatto quadrature rules)

We have seen in the lecture that Gaussian quadrature with s nodes allows to integrate exactly polynomials up to order $2s - 1$, and the quadrature nodes are inner nodes of the integration interval. However, it is often practical to include the extremes of the integration interval as quadrature nodes. When one of the two extremes is included, we have Gauss-Radau quadrature rules. When both extremes are included, we have Gauss-Lobatto rules. The price to pay is a slightly lower convergence rate.

Let $[a, b]$ be the integration interval. For the construction of Gauss-Radau quadrature rules with s nodes, we consider the monic polynomial

$$\omega(x) = q(x)(x - a),$$

where $q \in \mathbb{P}_{s-1}$. For a given weight function $w(x) > 0$, we write then

$$\int_a^b \omega(x)w(x)dx = \int_a^b q(x)(x - a)w(x)dx.$$

Similarly to the case of Gaussian quadrature, we can construct a family of orthogonal polynomials but now orthogonality is with respect to the modified weight $\hat{w}(x) = (x - a)w(x)$.

- (a) Show that $(p, q)_{\hat{w}} := \int_{-1}^1 p(x)q(x)\hat{w}(x)dx$ is a scalar product on the space of continuous functions on $[-1, 1]$ (so, in particular, for polynomials). Note that, in the lecture, we have used an analogous property but with the weight w when constructing Gaussian quadrature rules.
- (b) Use the scalar product defined in the previous task to construct a quadrature rule of order $2s - 1$ that has a as quadrature node. This is a Gauss-Radau quadrature rule.

Hint: You don't need to compute the quadrature nodes and weights explicitly. Proceed as we did in the lecture to specify the nodes for Gaussian quadrature, relying on nodes given as zeroes of an orthogonal polynomial.

- (c) Analogously, when we want to have both a and b as quadrature points, we use the monic polynomial $\omega(x) = q(x)(x - a)(x - b)$. Proceeding similarly to the previous tasks, construct a quadrature rule which has a and b as quadrature nodes and order $2s - 2$. This gives the family of Gauss-Lobatto quadrature rules (depending on the weight w).

Exercise 36 (Simpson's rule)

- (a) Compute an approximation to

$$\int_0^2 4x^3 + 3x^2 - 4x - 2 dx$$

using Simpson's rule. Which absolute error do you expect, compared to the exact integral? Check your expectation by computing analytically the value of the integral.

- (b) Use now the composite Simpson's rule with three equispaced intervals to approximate

$$\int_0^1 x \cos(x^2) dx.$$

Compute analytically the value of the integral, as well as the relative error of the approximation.